INDEX

NAME: Nabeela Meesal M STD.: _____ SEC.: _____ ROLL NO.: 241801179 SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	24.07.2026	Basic Network commands used in Linux & windows		
3	30.07.2026	Packet tracer tool installation and user interface		
2.	6.8.26	understand different types of cable, make in own Ethernet		
4.	6.8.26	Study of Local area network using a switch & Ethernet cable		
5.	6.8.26	Packet capture using wireshark		
6.	8.8.26	Error Detection and correction using hamingcode		
7.	13.8.26	Flow control of data link layer using Sliding window Protocol.		
9.	24.8.26	Implementation of Subnetting in cisco packet tracer		
10.a	30.8.26	Internetworking using Routers		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10.b.	15.9.26	wireless ROUTER, HDLP Server and Internet cloud		
11.	17.9.26	Routing at Network layer		
12.	22.9.26	Implement Echo client Server using TCP/UDP Sockets		
13.	26.9.26	Ping Program		
14.	30.9.26	Network Packet sniffer using Raw Sockets		
15.	5.10.26	Types of web logs using webalizer log analysis		

completed

Exp. No: 1
24/7/26

NETWORK COMPONENTS USED IN LINUX

Aim:

To study various network commands used in Linux and windows.

commands:

arp-a → shows IP address of the computer along with the IP and MAC Address.

hostname → displays name of the computer

ipconfig → displays configuration in the PC

nbstat -a → Helps to solve problems with NetBIO.

nslookup → displays detal DNS

pathping → Traces the route to the destination address then launches a 25 second test of each router along the way.

ping → Best way to test connectivity blw two node

```
#ping hostname
#ping ipaddress
#ping
```

Results:

Thus the network commands has been studied and executed successfully.

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Tcs>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens. The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.

Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.
```

ddresses

```
   > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

     CHANGE is used to modify gateway and/or metric only.

   > route DELETE 157.0.0.0
   > route DELETE 3ffe::/32
```

Exp No:3
30/7/26

PACKET TRACER TOOL INSTALLATION AND USER INTERFACE

Aim:

To study the Packet tracer tool installation and user interface.

1). From the network component box, click and drag all drop the below components.

a). 4 Generic PCs and one HUB

b). 4 Generic PCs and one switch

2). click on connections:

a). click on copper straight through cable

b). Select one of the PC and connect it to HUB using the cable.

3). Click on the PCs connected to the hub, go the desktop tab, click on IP configuration and enter the IP address.

a). Drag & Drop it on one of PC (source) and drag & drop it on another PC (destination)

4). Observe the flow of the PDU from source PC to destination PC by selecting one.

5). Repeat step 3 to step 5 for the PCs connected to the switch

6). Observe how hub & switch are forwarding the PDU and write your observation.

Result:

Thus, the study of packet tracer tool installation and user interface overview was sucessfully implemented.




Realtime
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete

Exp No:2
6/8/26

STUDY OF DIFFERENT TYPES OF NETWORK CABLES

Aim:

To study different types of network cables.

a). Understand different types of network cables

1) Unshielded Twisted Pair
2) Shielded Twisted Pair
3) Coaxial cable
4) Fibre optic cable

UTP

Category - 3	10 bps
category - 5	upto 100 mbps
Category 5e	1Gbps

Applications:

10 Base - T Ethernet
Fast Ethernet, Gigabit Ethernet
Fast Ethernet, Gigabit Ethernet

STP

Category 6, 6a	10 Gbbs	Gigabit Ethernet, 10 G Ethernet

SSTP

Category 2	10 Gbps	Gigabit Ethernet, 10G Ethernet (100m)

Advantages:

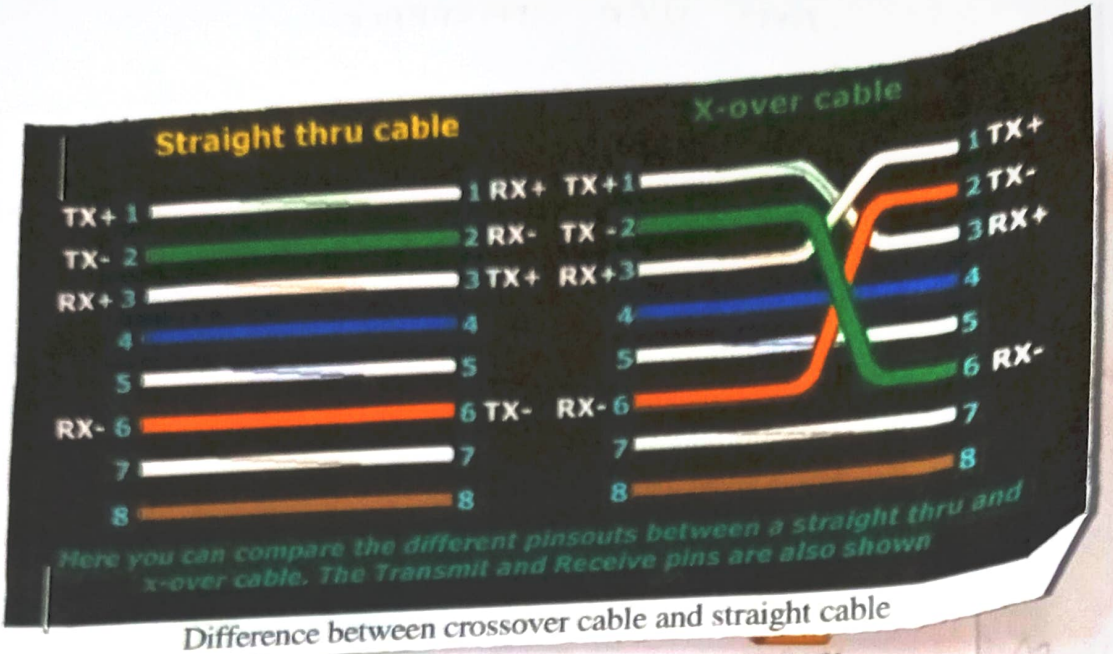
Sheilded
Faster than UTTP

Disadvantages:

Expensive
Greater Installation Effort

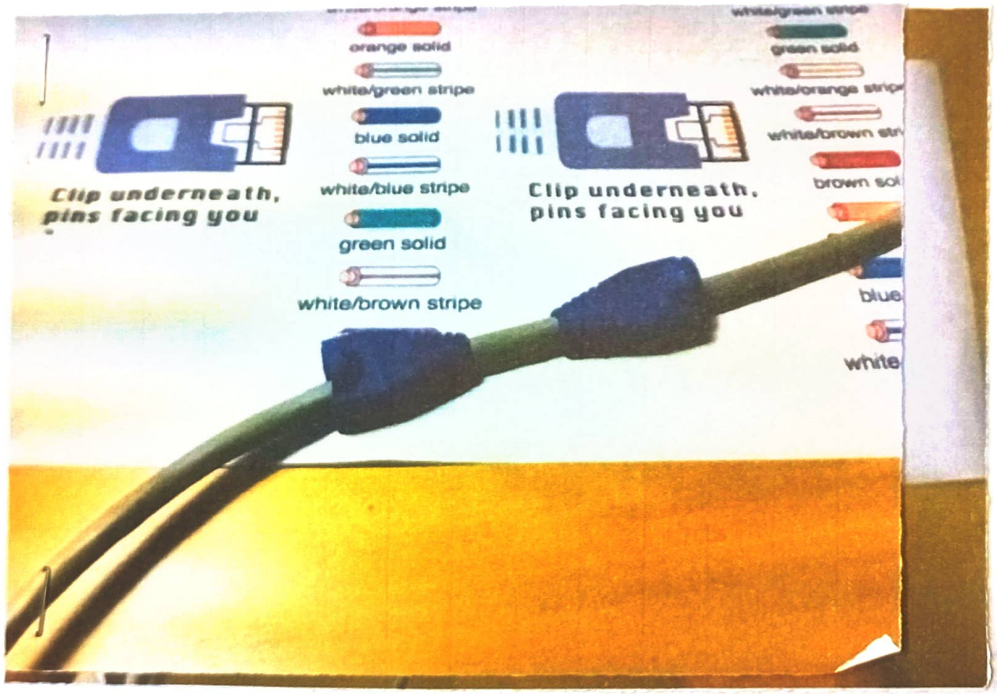
coaxial cable:

RG-6 RG 9 RG 11	10-100 mbps	Speed of signal in 500 m Television Network



Difference between crossover cable and straight cable





Advantages:

High bandwidth
Immune to Interference
Low loss Bandwidth
Versatile

Disadvantages:

Limited Distance
Cost
Size is bulkes

Fibre optics cable:

Single mode Bulk mode	100 Gbps	Maximum distance of fibre optic cable is around 100 metres.

Advantages:

High Speed
High Bandwidth
High Severity
Long Distance

Disadvantages:

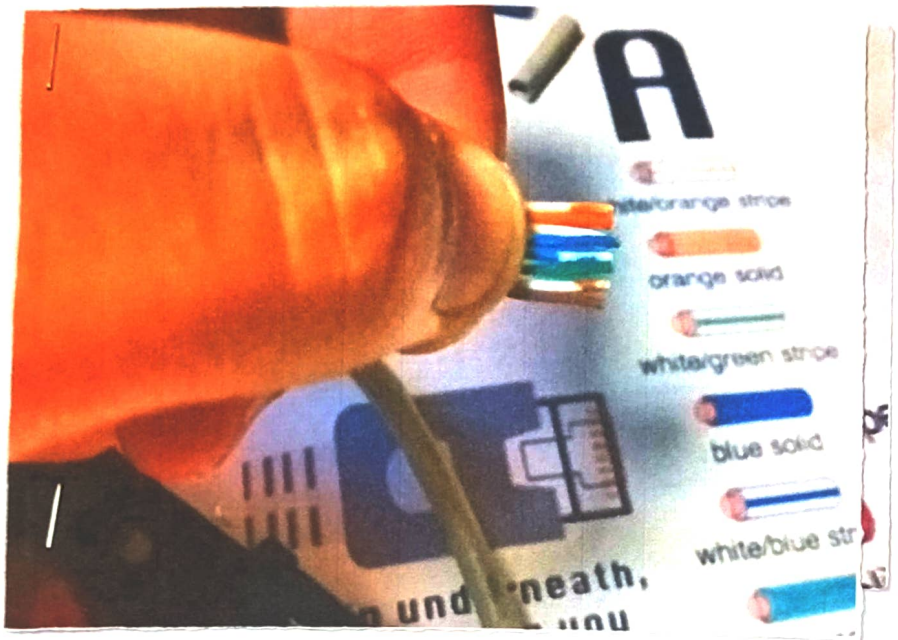
Expensive

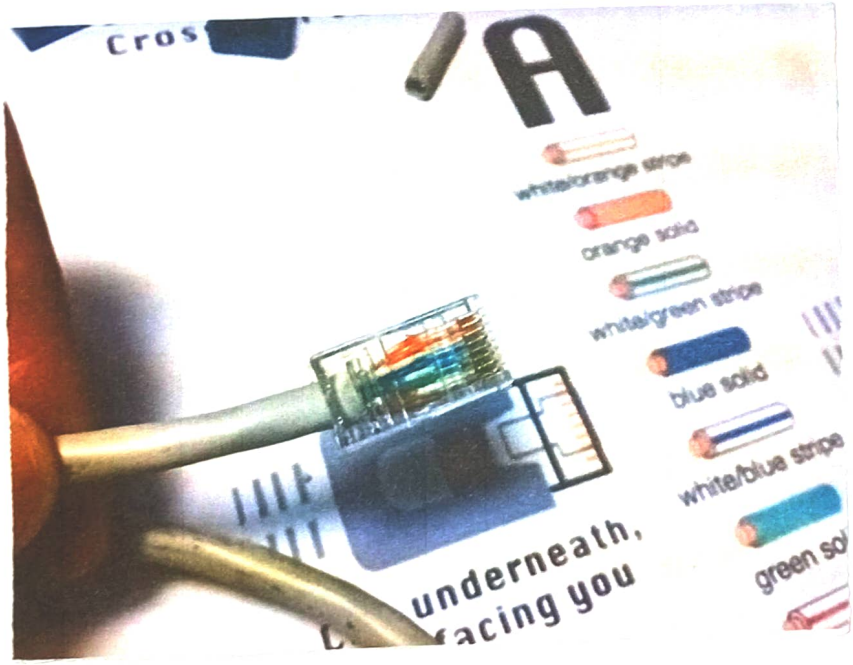
b). Make your own Ethernet:

Step:1 To start construction of the device, begin by threading shields onto the cable.
Step:2 Next step approximately 1.5cm from the cable shielding from both ends.
Step:3 After, you will need to untangle the wire, there should be four "twisted pairs".
Step:4 Once the order is correct, bunch them together in a line, and if there are way that stick and fastener than to wrote on even level.
Step:5 Next, push the cable right in, the clothes out the dual of the play needs to be just over the cable.
Step:6 Lastly, repeat the other end, to make a crossover cable

Result:

Thus study of different type of wires and working our own ethernet has been studied completely.


A
orange solid
white/green stripe
blue solid
white/blue str
underneath,


Cros
A
white/orange stripe
orange solid
white/green stripe
blue solid
white/blue stripe
underneath,
facing you

Exp.No:4
18/26

STUDY OF LOCAL AREA NETWORK USING A SWITCH & Ethernet CABLES

Aim:

TO set up and configure a local area network using a switch and ethernet cables and verify communication between connected computers

Procedure:

Plan the network by arranging 4 computer, one network switch and ethernet cable.

Connect each computer to the switch using ethernet cables.

Open control Panel → Network → Internet → Network computer.

Right click Ethernet → properties → TCP/IPV4 → Properties.

Select use the following IP address and assign

PC1 : 10.1.1.1, Subnet mask : 2250.0.0

PC2 : 10.1.1.2, Subnet mask : 255.0.0.0

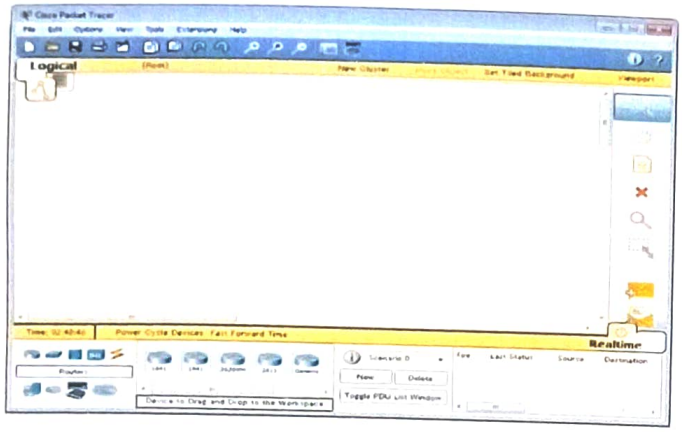
PC3 : 10.1.1.3, Subnet mask : 255.0.0.0

PC4 : 10.1.1.4, Subnet mask : 255.0.0.0

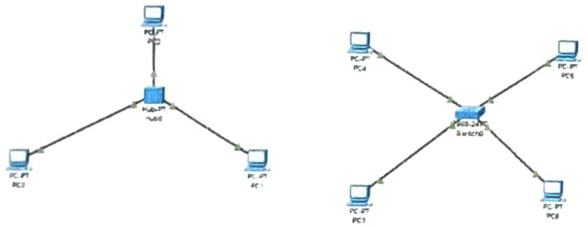
Configure the switch by connecting PC to a computer, accessing gt, web interface and assigning the switch IP as 10.1.1.5 with 255.0.0.0

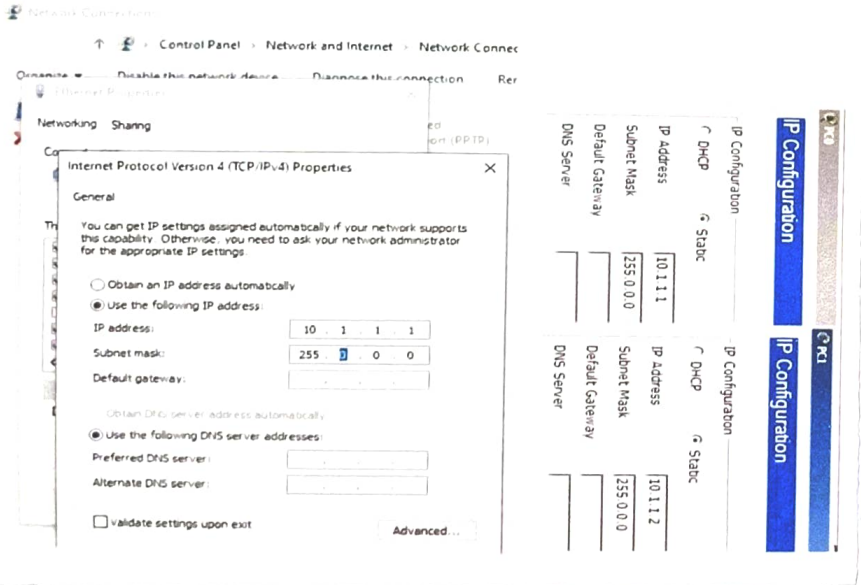
Result:

Thus, a local area network (LAN) was successfully setup and configured using a network switch and ethernet cables.


Logical
Realtime


Logical




Control Panel
Network and Internet
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address: 10 . 1 . 1 . 1
Subnet mask: 255 . 0 . 0 . 0
Default gateway:
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.1
255.0.0.0
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.2
255.0.0.0
PC0
PC1

Exp No:5
6/8/26

PACKET CAPTURE USING WIRESHARK

Aim:

To capture filter inspect and analyse network packets using the wireshark packet Analyser.

Algorithm:

Open Wireshark.
Select the active network interface.
Configure capture to stop after 100 packets.
Start Packet capture.
Apply display filters
Inspect packet details and view the flow graph
Save the captured packets.

Procedure:

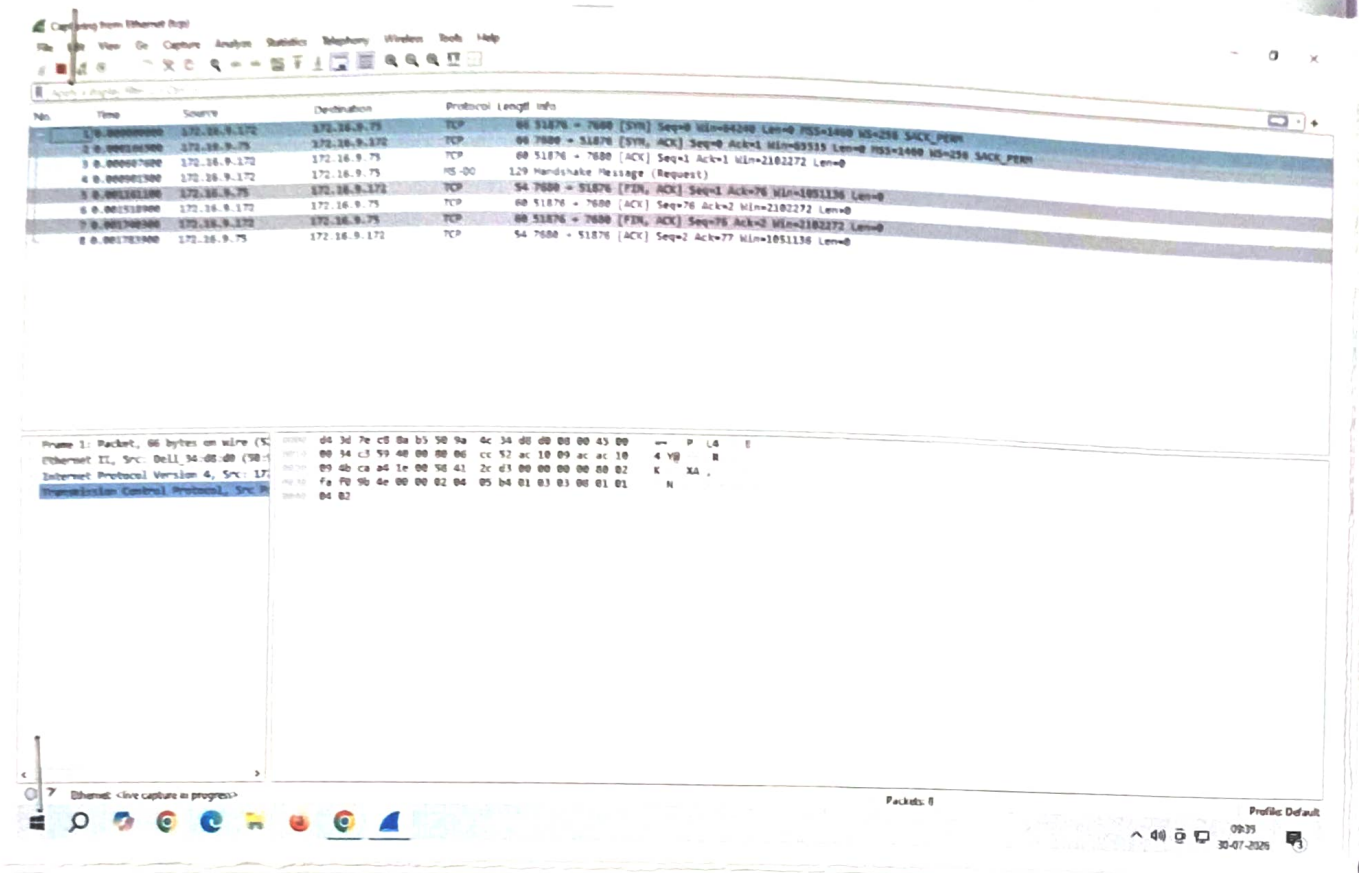
Launch wireshark and subset the active network interface
Go to capture -> options and set stop capture after 10 packets
click start to design packet capture

TCP/UDP	: tcp || udp
ARP	: arp
DNS	: dns
HTTP	: http
DHCP	: bootp

Inspect the filtered packets in the packet list, details and bytes panes.
Save the capture file or future analysis.

Result:

Successfully captured 100 network packets. applied display filters for TCP, UDP, ARP, DNS, HTTP, IP, ICMP, DHCP analysed packets details using wireshark.


Capturing from Ethernet
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
1 0.000000000 172.16.9.172 172.16.9.75 TCP 66 51876 → 7680 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2 0.000106500 172.16.9.75 172.16.9.172 TCP 66 7680 → 51876 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3 0.000607000 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [ACK] Seq=1 Ack=1 Win=2102272 Len=0
4 0.000901900 172.16.9.172 172.16.9.75 MS-DO 129 Handshake Message (Request)
5 0.001101100 172.16.9.75 172.16.9.172 TCP 54 7680 → 51876 [FIN, ACK] Seq=1 Ack=76 Win=1051136 Len=0
6 0.001518900 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [ACK] Seq=76 Ack=2 Win=2102272 Len=0
7 0.001708300 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [FIN, ACK] Seq=76 Ack=2 Win=2102272 Len=0
8 0.001783900 172.16.9.75 172.16.9.172 TCP 54 7680 → 51876 [ACK] Seq=2 Ack=77 Win=1051136 Len=0
Frame 1: Packet, 66 bytes on wire (5
Ethernet II, Src: Dell_34:d8:d0 (50:
Internet Protocol Version 4, Src: 17
Transmission Control Protocol, Src P
0000 d4 3d 7e c8 0a b5 50 9a 4c 34 d8 d0 08 00 45 00
0010 00 34 c3 59 40 00 80 06 cc 52 ac 10 09 ac ac 10
0020 09 4b ca a4 1e 00 58 41 2c d3 00 00 00 00 80 02
0030 fa f0 9b 4e 00 00 02 04 05 b4 01 03 03 08 01 01
0040 04 02
Ethernet: <live capture in progress>
Packets: 8
Profile: Default
09:39
30-07-2026

Exp No 6
8.8.26

ERROR DETECTION AND CORRECTION USING HAMMING CODE

Aim:

To write a program to implement error detection and corrector using the hamming code.

Algorithm:

1. Start the program
2. Read the number of 4-digit data block.
3. For each digit 4 bit data block (4) perform hamming.

$P_1 = d1 \text{ XOR } d2 \text{ XOR } d4$

$P2 = d1 \text{ XOR } d3 \text{ XOR } d4$

$P3 = d2 \text{ XOR } d3 \text{ XOR } d4$

4. Simulate transmission by introducing a single bit error.
5. $C1 = P1 \text{ XOR } d1 \text{ XOR } d4$; $C2 = P2 \text{ XOR } d1 \text{ XOR } d3 \text{ XOR } d4$; $C4 = P1 \text{ XOR } d2 \text{ XOR } d3 \text{ XOR } d4$
6. Compute error position = $C4C_2C_1$
7. If error position = 0, no error has occured.
8. Extract the original data bits from the corrected 7 bit.
9. Repeat Step 3 to 8 for all n-blocks in the data
10. Stop the Program.

Program:

```
def encoded (d):
    h = [0]* 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[0] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1 * 1 + (2*2 + c4*4
    if error_pos != 0
        h[error pos] ^ = 1
```

```
        else:
            print("No error")
        return error_pos
def main():
    n = int(input("Enter number:"))
    for b in range(1, n+1):
        d = list(map(int, input("Enter for data bits").split())
        h = encode(d)
        print(f"Original Data Bit: {to_str(d)}")
        print(f"Encoded Hamming code: {to_str(a)}")
        if 1<=pos<=7:
            h[pos-1] = 1
            print(f"Transmitted code: {to_str(h)}")
        else:
            print(f"Transmitted code: {to_str(h)}")
    Printf("Recieved code: {to_str(h)}")
    detect_and correct(h)
    print(f"corrected code: {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    print(f"Recured Data Bit: {to_str(rd)}")
    if __name__=="__main__":
        main()
```

Result:

The program to implement error detection and correction using the Hamming code concept was successfully executed and verified.

Output:

```
Enter number of 4 bit data blocks: 2
--Block 1--
Enter 4 data bits: 1011
original hamming code: 1011
Encoded hamming code: 0110011
Enter bit portion to introduce error: 3
Transmitted code: 0100111
Recieved code: 0100011
corrected code: 0110011
recovered code data bits: 1011
--Block 2--
Enter 4 data bits: 0110
original data bits: 0110
Encoded hamming code: 1100110
Enter bit Position to introduce error: 6
Transmitted code: 1100100
Recieved code: 1100100
Error detected bit position: 6
Corrected code: 1100110
Recovered data bits: 0110
```

Ex. No: 7
03.9.26

FLOW CONTROL OF DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

Aim:

To write a program to implement flow control at the data link layer using sliding window Protocol.

Algorithm:

1. Start the Program
2. Read the total no. of frames to be sent(N). the sender is window size (w).
3. Initialize true = 0
4. Repeat while base < N
   a. Determine the current under window as the range [base, base+w-1]
   b. Send all the frames with sequence number from next req.
   c. If the wait frame lies within the frames just sent.
      i). The receiver discards the dort frame and any subsequent.
      ii). In sender times out and retransmits all frames
   d. Else
      i). The reciever accepts each frame in order and sends a cumulative acknowledgement
      ii). On recieving the Ack for the lost frame, the sender slids the window forward.
5. Repeat steps until all N frames transmitted
6. Stop

Program:

```
def simulated(total_frames window_size, bits_frame
    base = 0
    next_req = 0
    while base < total_frames:
      window_end = min(base + window_size - 1, total_frames
    Print(f" sender window: [frame {base} - frame {window_end
while next_seq < base + window_size and next_seq < total_frames
    if next_seq == lost frame:
        Print(f"[frame {next_seq} LOST in transit:")
    next_seq += 1
```

Output:

Enter total no. of frames to end : 8
Enter sender window size : 4
Enter frame no to simulate as LOST : 5
Sender window : [frame 0 - frame 3]
Node A -> Sender frame 0
Node A -> Sending frame 1
Node A -> Sending frame 2
Node A -> Sending frame 3
Node B -> Received frame 0, Sending ACK 0
Node B -> Received frame 1, Sending ACK 1
Node B -> recieved frame 2, Sending ACK 2
Node B -> recieved frame 3, Sending Ack 3
Sender window : [frame 4 - frame 7]
Node A -> Sending frame 4
Node A -> Sending frame 5
Node A -> Sending frame 6
Node B -> frame 5 missing
Node A -> Timeout : recieving window starting fr
frame 5
Sender window : [frame 4 - frame 7]
Node A -> Sending frame 5
Node A -> Sending frame 6
Node A -> Sending frame 7
Node B -> recieved frame 4, Sending ACK 4
Node B -> recieved frame 5, Sending ACK 5
Node B -> recieved frame 7, Sending ACK 7
Node A -> ACK 7 recieved Sliding window forwa

All 8 frames sucessfully transmitted from node A to node B.

```
if lost_frame is not None and base <= lost_frame < base + window_size:
    print(f"Node B --> Frame {lost_frame} missing. Discarding
      out of order frames after it.")
    next_seq = lost_frame
    lost_frame = None
for f in range(base, next_seq):
    print(f"Node A --> Ack {next_seq - 1} received. Sliding
                                    window forward:")
def main():
    total_frames = int(input("Enter total no. of frames
                                    to send:"))
    lost_frame = int(input())
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)
if __name__ == "__main__":
    main()
```

Result:

The program to implement flow control at the data link layer using the sliding window protocol was successfully executed and verified by simulating the flow of frames between two nodes.

Ex.No:9
24.9.26

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER

Aim:

To implement classless IP subnetting using the cisco Packet tracer simulator and test connectivity between different subnets.

Requirements:

Cisco Packet Tracer
Routers
Switches
PCs
Ethernet cables

Procedure:

1. Creating the Network Topology

Open cisco packet tracer
create a new workspace
Add the required routers, switches and PCs.
connect the device using appropriate Ethernet cable

2. IP Addressing

LAN1 R1 G0/0 192.168.1.1/27
LAN2 R1 G0/1 192.168.2.1/27
LAN3 R2 F0/0 192.168.3.1/27
LAN4 R2 F0/1 192.168.4.1/27

For example,

IP Address : 192.168.1.11
Subnet mask : 255.255.255.224
Default gateway : 192.168.1.1

3. Configuring Router R1

Open R1 -> CLI and enter:

```
enable
configure terminal
interface gigabit Ethernet 0/0
IP Address 192.168.1.1.255.255.224
no Shutdown
exit
```

4. Configuring Router R2

```
enable
configuring terminal
interface fast Ethernet 0/0
IP Address 192.168.3.1.255.255.224
no shutdown
exit
```

5. Configuring the switches:

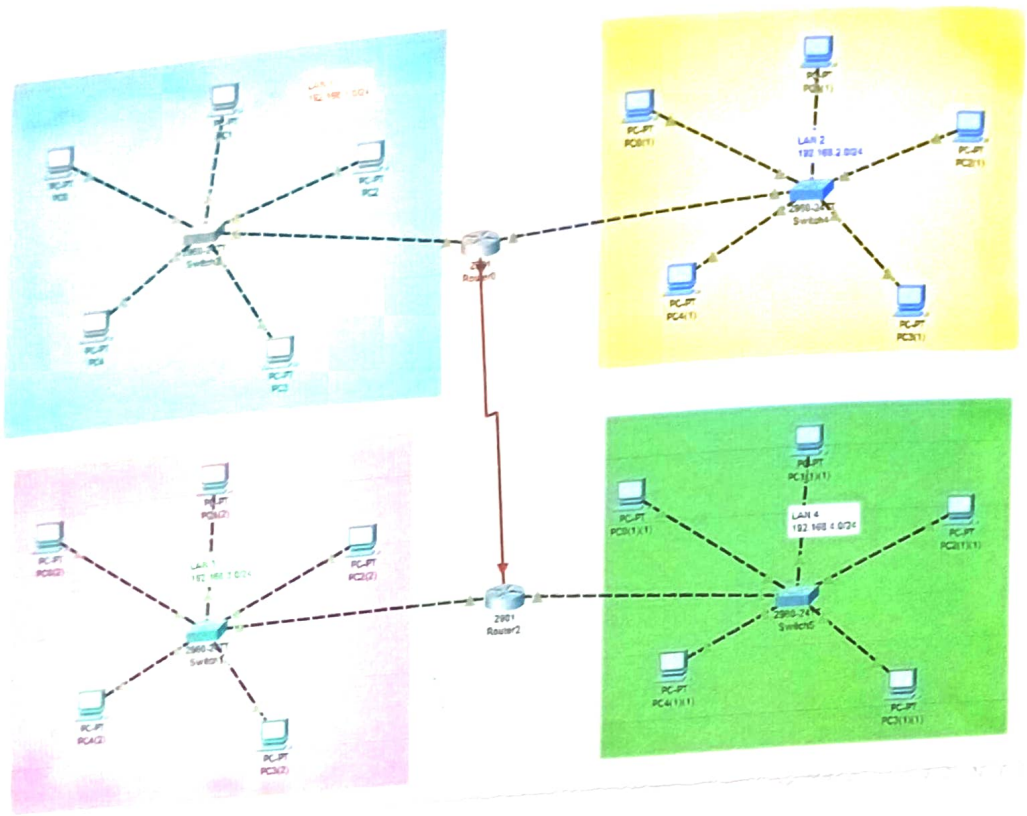
For S1:

```
enable
configure terminal
interface fast Ethernet 0/1
 switch Port mode access
 Exit
```

6. Configuring the PCs.

Open each PC -> Desktop -> IP configuration and enter the appropriate IP address subnet mask and default gateway.

PC	IP Address	Subnet Mask	default Gateway
PC1	192.168.1.11	255.255.224	192.168.1.1
PC2	192.168.1.12	255.255.224	192.168.1.1


LAN 2
192.168.2.0/24
2901
Router0
2901
Router2
LAN 4
192.168.4.0/24

Testing the Network:

1. Open PC -> Desktop -> command prompt

ping 192.168.1.1

Test connectivity blw PCs in the Subnet: 192.168.1.12

Result:

Implemented classes IP subnetting using the CISCO Packet tracer simulator is successfully completed.

Ex.No:10a
2.9.26

INTERWORKING USING ROUTERS

Aim:

To design and configure a simple internetwork using a router and two in CISCO Packet Tracer and test connectivity using PPU.

Step 1: Configure Router 1

1). Open Cisco Packet Tracer

2). Select Router -> Router 1

3). OPEN CLI and Press enter

4). Enter

enable

config t

5). Configure FastEthernet 0/0:

Interface FastEthernet 0/0

IP address 192.168.10 255.255.255.0

No shutdown

6). Configure FastEthernet 0/0:

Interface Fast Ethernet 0/1

IP address 192.168.10 255.255.255.0

No shutdown

Step 2: Configure PC0

Go to PC0 -> desktop -> IP configuration

IP address: 192.168.10.2

Subnet Mask: 155.255.255.0

Default Gateway: 192.168.10.1

Step 3: Configure PC1

Go to PC1 -> desktop -> IP configuration

IP address: 192.168.20.2

Subnet mask : 255.255.255.0

Default Gateway : 198.168.20.1

Step 4 : Connect the devices

Use copper Straight-Through cable

PC0 Fast Ethernet 0 -> Router (Fast Ethernet 0/0

PC1 Fast Ethernet 0 -> router (Fast Ethernet 0/1

Step 5 : Test connectivity

Select Simulation mode

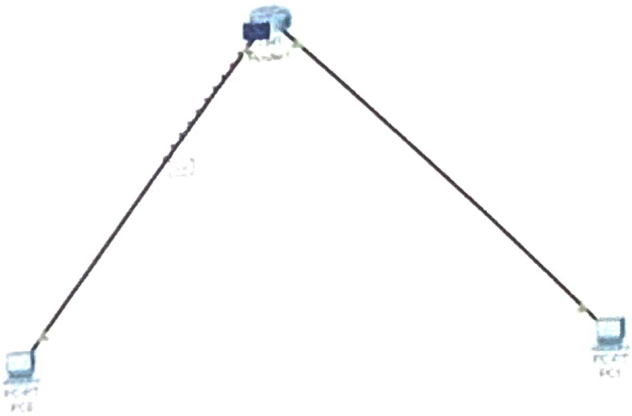
Chose Add simple PDU

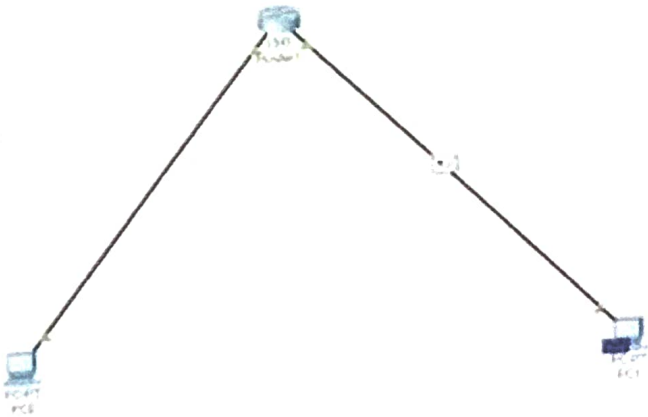
Click PC0 -> PC1

If PDU is sucessful, the two network are communicating the router.

Result :

The Internetwork was sucessfully designed and configured and connectivity between PC0 and PC1 was verified.





No: 10b
9-26

HIRELESS ROUTER, DHCP SERVER AND INTERNET CLOUD

Aim:

To design and configure an internetwork using a wireless router, DHCP server and Internet cloud and verify connectivity.

Step 1: Build the Network

Add:

- PC
- Wireless Router
- laptop
- Cable Modem
- Internet cloud
- cisco.com server

Step 2: connect Devices

Use the appropriate cables

PC -> wireless Router -> cable Modem : copper Straight - through

Wireless Router -> cable Modem : copper Straight - through

cable Modem -> Internet cloud : coaxial cable

Internet cloud -> cisco.com server : copper Straight - through

Step 3: configure wireless Router

Open wireless Router -> GUI

Go to windows

Set DNS server as 208.67.220.200.220

click save settings

Step 4: Configure cisco.com Server

DHCP

DHCP: ON

Pool Name: DHCP Pool

Default Gateway: 208.67.220.220

Starting IP: 208.67.220.1

Maximum users: 50

Click add

Step 5: Test connectivity

Open PC → Desktop → command prompt

Type: ipconfig/release

Then: ipconfig/renew

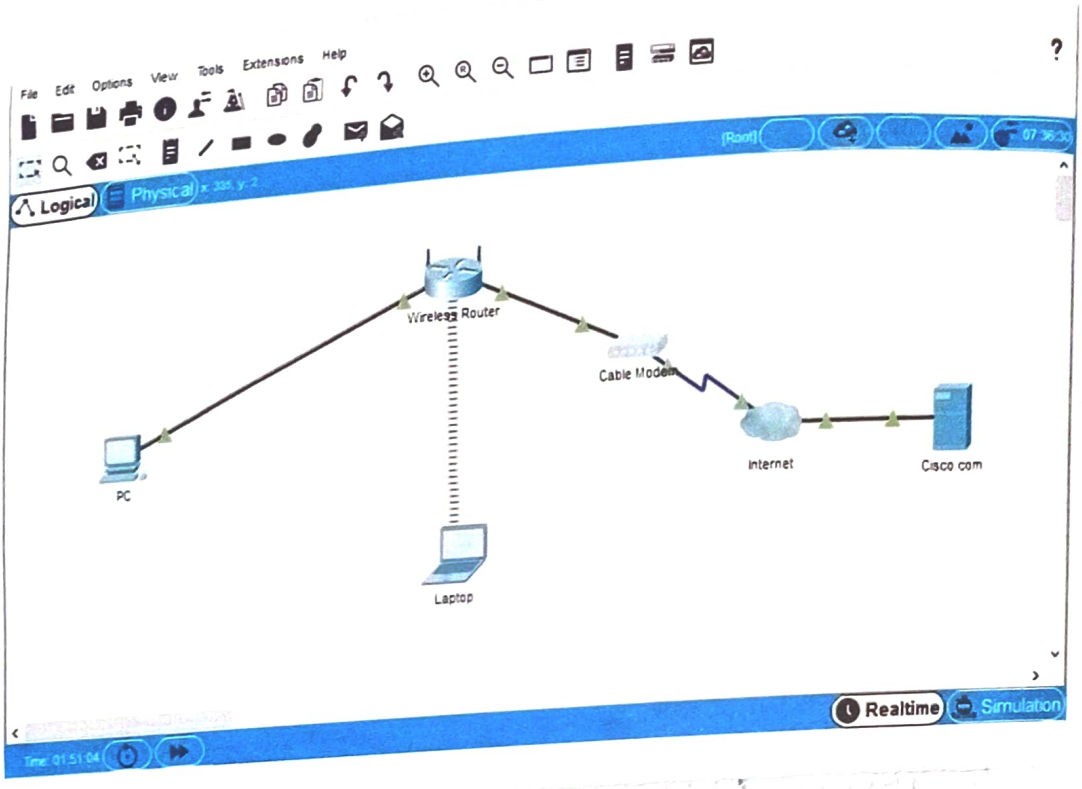
test the server: ping cisco.com





Result:

The wireless internet work was successfully designed using a wireless router DHCP server and internet cloud and connectivity was verified using the ping command.


File Edit Options View Tools Extensions Help
Logical
Physical
Wireless Router
Cable Modem
PC
Laptop
Internet
Cisco com
Realtime
Simulation

EX.NO: 11
17.9.26

ROUTING AT NETWORK LAYER

a). Simulate static routing protocol configuration using cisco packet Tracer.

Aim:

To simulate and configure static routing between multiple networks using cisco packet tracer, configure primary and backup static routers using administrative distance

Requirements:

cisco packet tracer

Three-routers - Router 0, Router 1, 2

PCs/end devices

Properly assigned IP address

Observation:

A network topology consisting of three routers, Router 1, 2 was created in cisco packet tracer, different networks were connected and were assigned suitable IP address.

Router 1:

Router 1 was configured with static route to the 10.0.0.0/8 to 40.0.0/8 networks. The route through Router 0 was assigned a lower administrative distance Primary route The configuration was verified using show IP route static.

Router 2:

Router 2 was configured with static routes to the 10.0.0/8 and 30.0.0/8 networks. The routing table and verified using show ip route static.

Backup route verification:

The main link between Router 0 and was disconnected to simulate a link failure connectivity was verified using Pink and trace, confirming that the backup route was working successfully.

Verification:

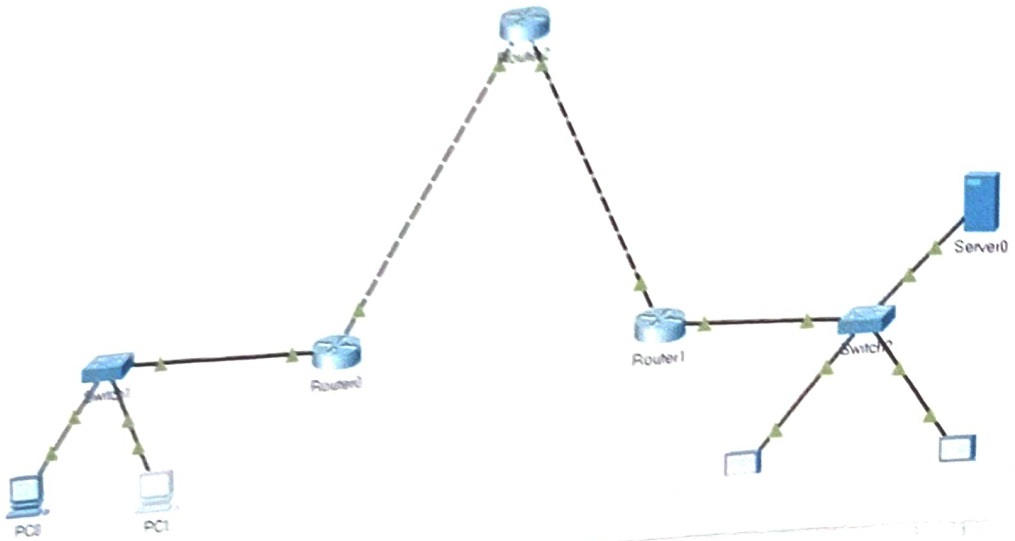
Show IP route static. The routing table showed the configured static routes and their Administrative using the Ping command successful ping between source and destination networks.

Deleting a Static Route:

Show IP route static. After deleting the Main route, the backup route becomes the active if the alternate path is available.

Result:

Thus, static routing was successfully configured and simulated using cisco Packet tracer. The Primary and backup routes were configured using different.


Server0
Router1
PC0
PC1

EX.NO:12
22-9-26

IMPLEMENT ECHO CLIENT SERVER USING TCP/UDP SOCKETS

Aim:

To write and implement an echo client server program using TCP sockets in

Result:

Thus, the echo client server was successfully implemented using sockets.



```
        exit(EXIT_FAILURE);
    }

    // 2. Bind socket
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // 3. Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }

    printf("[TCP Server] Waiting for incoming connections on port
%d...\n", PORT);

    // 4. Accept connection
```

Ex NO: 13
26.9.26

PING PROGRAM

Aim:

To implement a Ping Program using Python raw sockets to check the reach of a host and calculate the round trip time

Result:

Thus Ping Program using Python to check the reachability of a host in calculated successfully.

```
            // Extract IP header to inspect TTL and payload
            struct ip *ip_hdr = (struct ip *)recv_buf;
            int ip_hdr_len = ip_hdr->ip_hl * 4;
            struct icmp *reply_icmp = (struct icmp *)(recv_buf +
ip_hdr_len);

            // Calculate RTT in milliseconds
            double rtt = (tv_end.tv_sec - tv_start.tv_sec) * 1000.0 +
                    (tv_end.tv_usec - tv_start.tv_usec) / 1000.0;

            if (reply_icmp->icmp_type == ICMP_ECHOREPLY) {
                printf("%d bytes from %s: icmp_seq=%d ttl=%d
time=%.2f ms\n",
                    bytes_rcvd - ip_hdr_len,
                    inet_ntoa(from_addr.sin_addr),
                    reply_icmp->icmp_seq,
                    ip_hdr->ip_ttl,
                    rtt);
            }
        }
        sleep(1); // Wait 1 second between requests
    }

    close(sockfd);
```

Ex.No: 14
Date: 7.9.26

NETWORK PACKET SNIFFER USING RAW SOCKETS

Aim:

To write and implement a network pocket sniffer using Raw sockets in to capture and parse IP, TCP and UDP

Result:

Thus the network sniffer using raw sockets is implemented sucessfully to capture and parse IP, TCP and UDP.

```
printf("IP Header:\n");
printf("   |-IP Version        : %d\n", (unsigned int)ip_hdr->version);
printf("   |-Header Length     : %d Bytes\n", ip_hdr_len);
printf("   |-Time To Live (TTL): %d\n", ip_hdr->ttl);
printf("   |-Protocol          : %d\n", ip_hdr->protocol);
printf("   |-Source IP         : %s\n", inet_ntoa(source.sin_addr));
printf("   |-Destination IP    : %s\n", inet_ntoa(dest.sin_addr));

// Handle TCP Packets
if (ip_hdr->protocol == IPPROTO_TCP) {
   struct tcphdr *tcp_hdr = (struct tcphdr *)(buffer + ip_hdr_len);
   printf("TCP Header:\n");
   printf("   |-Source Port       : %u\n", ntohs(tcp_hdr->source));
   printf("   |-Destination Port  : %u\n", ntohs(tcp_hdr->dest));
   printf("   |-Sequence Number   : %u\n", ntohl(tcp_hdr->seq));
   printf("   |-Acknowledge No    : %u\n", ntohl(tcp_hdr->ack_seq));
   printf("   |-SYN Flag          : %d\n", tcp_hdr->syn);
   printf("   |-ACK Flag          : %d\n", tcp_hdr->ack);
   printf("   |-FIN Flag          : %d\n", tcp_hdr->fin);
}
// Handle UDP Packets
else if (ip_hdr->protocol == IPPROTO_UDP) {
   struct udphdr *udp_hdr = (struct udphdr *)(buffer + ip_hdr_len);
   printf("UDP Header:\n");
   printf("   |-Source Port       : %u\n", ntohs(udp_hdr->source));
   printf("   |-Destination Port  : %u\n", ntohs(udp_hdr->dest));
```

Ex.No:
5.10-26

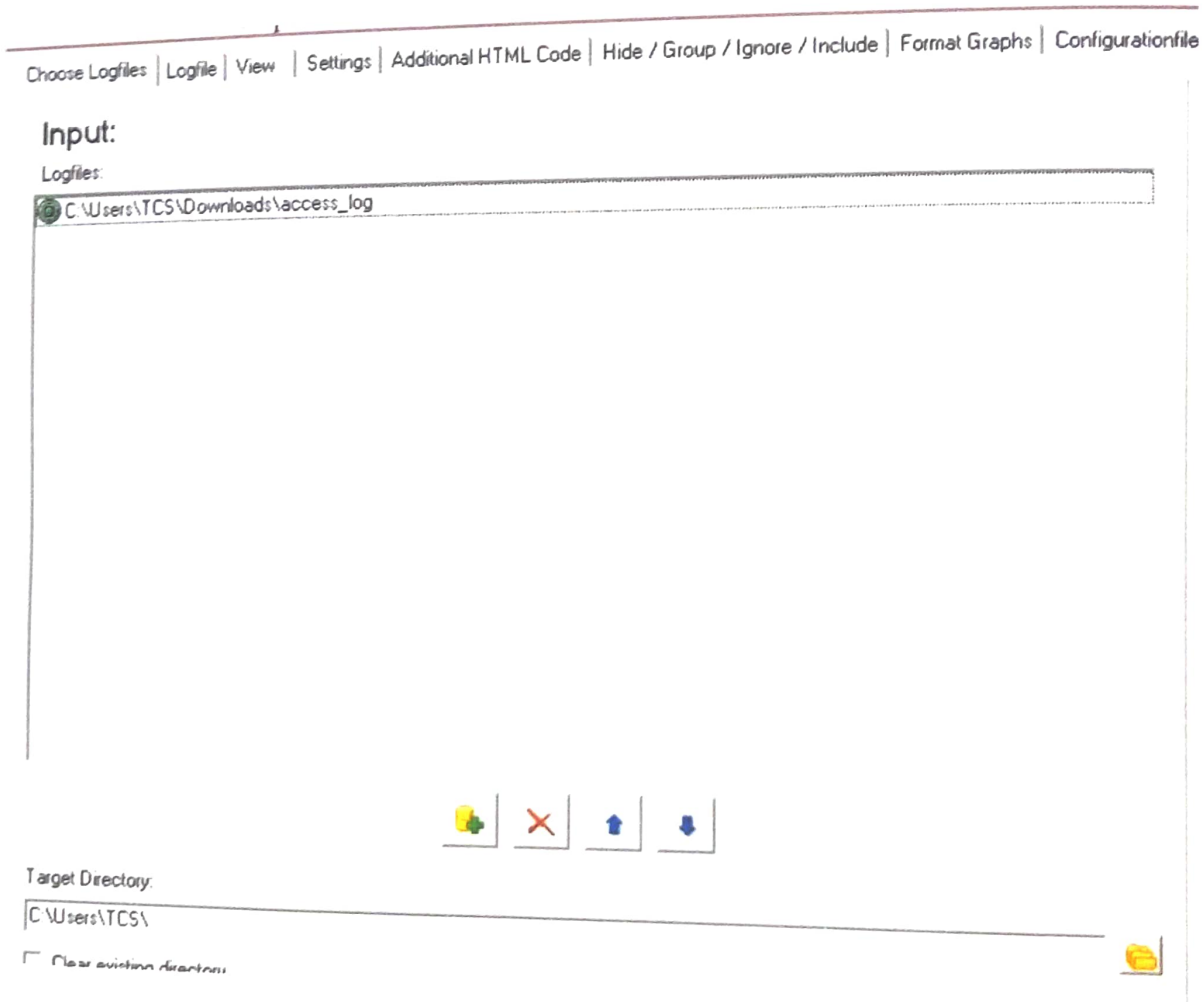
WEB LOGS USING WEBALIZER LOG ANALYSIS

Aim:

To analyse different types of web pages logs using the webalizer log analysis tool.

Result:

Thus different types of web logs using the webalizer log analysis is analysed successfully.


Choose Logfiles | Logfile | View | Settings | Additional HTML Code | Hide / Group / Ignore / Include | Format Graphs | Configurationfile
Input:
Logfiles:
C:\Users\TCS\Downloads\access_log
Target Directory:
C:\Users\TCS\